

The image features a blue gradient background with white circuit-like lines in the corners. These lines consist of straight segments and small circles, resembling a stylized electronic circuit or data network. They are positioned in the top-left, top-right, bottom-left, and bottom-right corners, framing the central text.

GLIDERECORD

TABLE OF CONTENTS

- Introduction to API
- GlideRecord API
- Methods of GlideRecord introduction
- Stages of GlideRecord
- GlideRecord addQuery() method
- GlideRecord addEncodedQuery() method
- Accessing a Record's field
- GlideRecord get() method
- GlideRecord CRUD operations
- GlideAggregate

API (APPLICATION PROGRAMMING INTERFACE):

- An API is building block to help the developer, it can define how to interact with a service or system, Web **APIs** provide machine-readable data and functionality transfer between web-based systems which represent client-server architecture.

- In ServiceNow if we want to create an new API

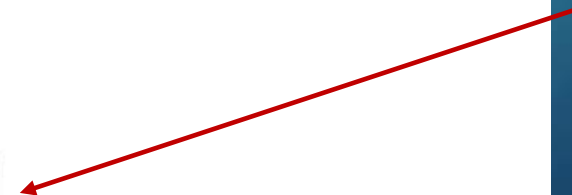
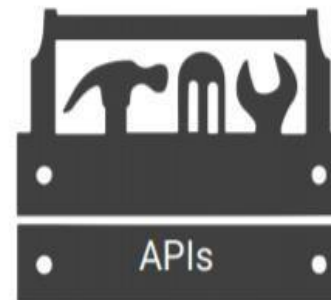
Navigate to System Web Services > Scripted REST **APIs** & Click New.

- ServiceNow APIs are:

- GlideRecord
- GlideSystem
- GlideUser
- GlideForm
- GlideAjax
- GlideDateTime
- Many more

Analogy: Building A House

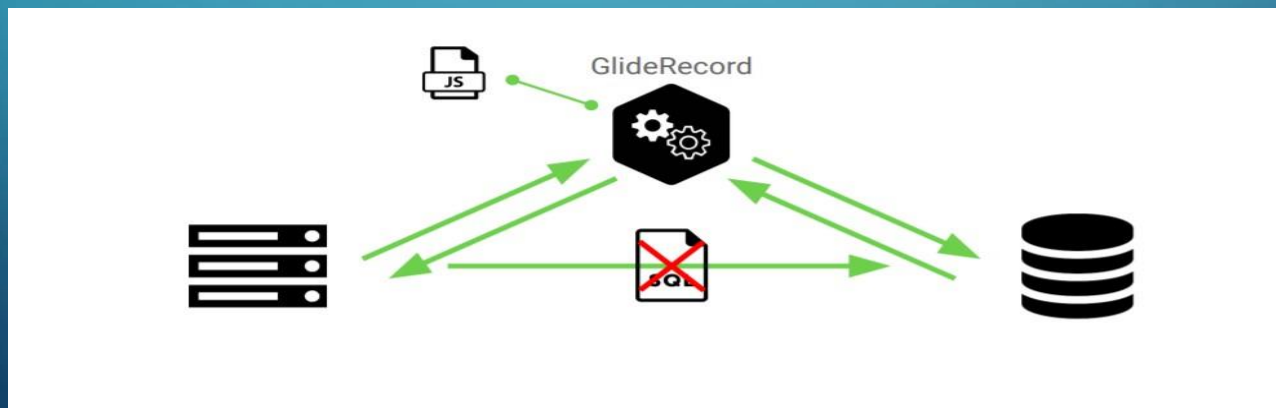
- Goal: Build a house
- A house needs materials (scripting locations)
 - Foundation - Business Rules
 - Doors - Client Scripts
 - Electrical - UI Actions
 - Materials - Script Includes
 - Etc.
- Tools (APIs)
 - Hammer - GlideRecord
 - Screwdriver - GlideSystem
 - Wrench - GlideUser
 - Etc.



GlideRecord:

GlideRecord is a java class & used for **database operations(CRUD)**.GlideRecord can be used in server side scripting as well as client side scripting. The client-side GlideRecord API enables the use of some GlideRecord functionality in client-side scripts, such as client scripts and UI policy scripts. It is **most commonly used API**.

- Generates **SQL**.
- It has two stages i.e. **Building query & processing records**.
- Topics associated with this API are table,query,records and fields.



GlideRecord API Diagram



Note: Ovals represent properties or topics associated with API, while rounded rectangles represent methods

Methods OF GlideRecord :

- Glide record for the specified table is created by using the following format

GlideRecord(String tableName)

Ex: `var incidentGR = new GlideRecord('incident');`

- addQuery()
- next()
- addEncodedQuery()
- setLimit()
- getRowCount()
- hasNext()
- addNotNullQuery()
- deleteRecord()
- query()
- newRecord()
- order()
- update() etc..

GlideRecord API Mapping



Common GlideRecord Methods

- query()
- newRecord()
- insert()
- update()
- deleteRecord()
- addQuery()
- addEncodedQuery()
- hasNext()
- next()
- get()
- orderBy()
- orderByDesc()
- canCreate()
- canWrite()
- canRead()
- canDelete()

➤ These are few of the most commonly used Gliderecord Methods.

❖ **Note::** Any ServiceNow table will be queried with GlideRecord

➤ This is how the GlideRecord Methods are categorized into **Chain Methods** where we can those methods together and **Encoded Query** where we build query by using graphical user interface(**GUI**) and by copying query & will be passing in this method.

1 Chain Methods

- addQuery()
- addOrCondition()
- addNullQuery()
- addNotNullQuery()
- addActiveQuery()
- addInactiveQuery()

2 Encoded Query

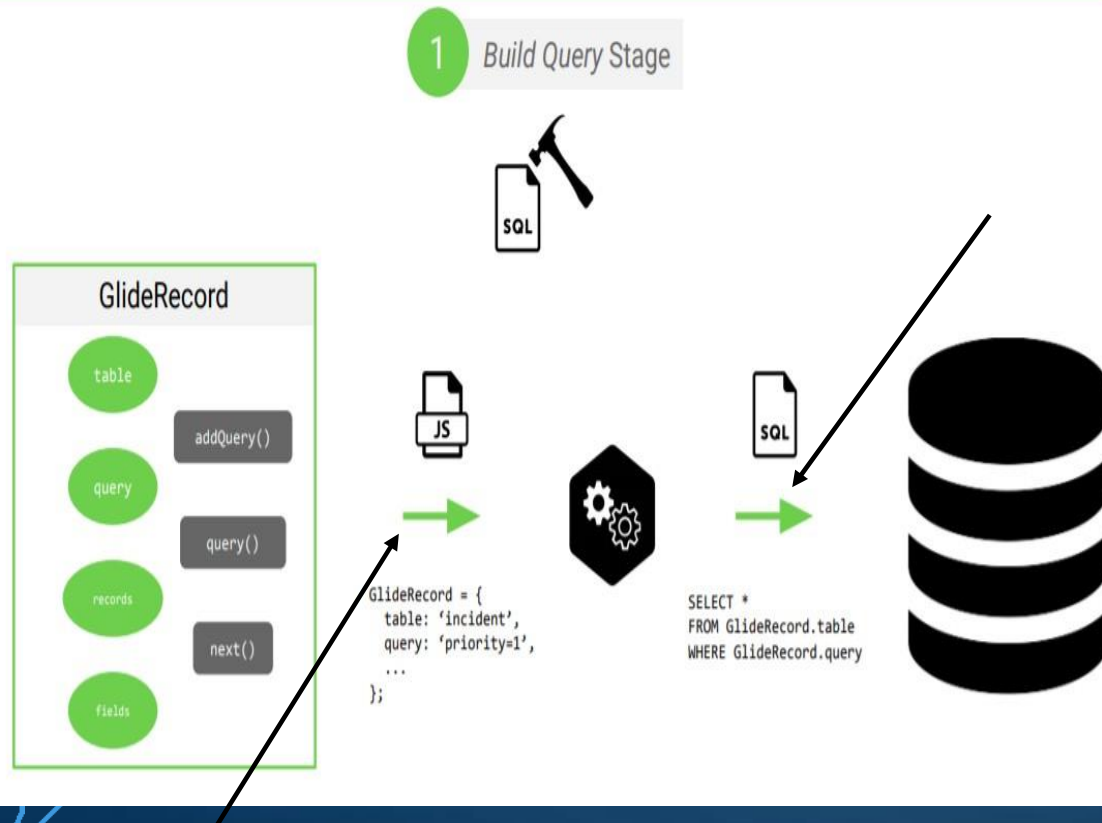
- addEncodedQuery()

Stages of GlideRecord :

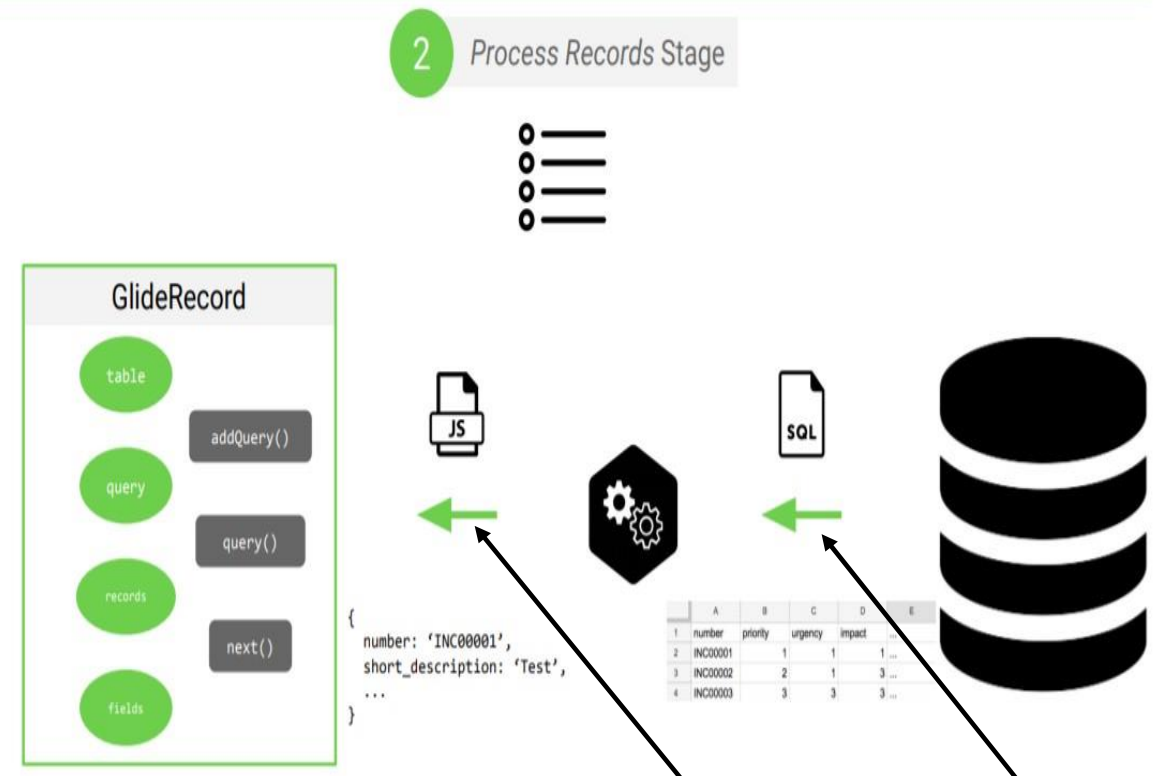
There are two stages

- Building a query.
- Process Records.

GlideRecord Stage 1: Building Query



GlideRecord Stage 2: Process Records



GlideRecord addQuery() method:

- Build a search query and return the rows that match the request.
- This method is similar to the "where" clause as in SQL.
- It accepts two or three arguments.

1. Two arguments: addQuery(String name, Object value)

When addQuery() is called with only two parameters, table field and comparison

ex: `rec.addQuery('active', true);`

In this case the operator is assumed to be "equal to".

2. Three arguments: addQuery(String name, String Operator, Object value)

In this case the queries are AND'ed. If any of the query statements need to be OR'ed, use the GlideQueryCondition method `addOrCondition`.

Example:

```
rec.addQuery('sys_created_on', ">", "2010-01-19 04:05:00");
```

```
now_GR.addQuery('number', 'IN', 'INC00001, INC00002');
```

2 Arguments

```
1 var incidentGR = new GlideRecord('incident');  
2 incidentGR.addQuery('short_description', 'Test');
```

field_name

field_value

3 Arguments

```
1 var incidentGR = new GlideRecord('incident');  
2 incidentGR.addQuery('short_description', 'CONTAINS', 'Test');
```

field_name

operator

field_value

Numbers

= >= >
!= <= <

Strings

= != STARTSWITH DOES NOT CONTAIN
IN NOTIN ENDSWITH CONTAINS
INSTANCEOF



Note: '=' is assumed if there are only 2 arguments

Name	Type	Description
name	String	Table field name.
operator	String	Query operator. The available values are dependent on the data type of the value parameter. Numbers: = != > >= < <= Strings (must be in upper case): = != - IN - NOT IN - STARTSWITH - ENDSWITH - CONTAINS - DOES NOT CONTAIN - INSTANCEOF
value	Object	Value on which to query (not case-sensitive).

GlideRecord addEncodedQuery() method:

Adds an encoded query to other queries that may have been set.

An incorrectly constructed encoded query, such as including an invalid field name, produces an invalid query.

When the invalid query is run, the invalid part of the query condition is dropped, and the results are based on the valid part of the query, which may return all records from the table. Using an insert(), update(), deleteRecord(), or deleteMultiple() method on bad query results can result in data loss.

addEncodedQuery(String query)

```
EX:var queryString = "priority=1^ORpriority=2";
    var now_GR = new GlideRecord('incident');
    now_GR.addEncodedQuery(queryString);
    now_GR.query();
    while (now_GR.next()) {
        gs.print(now_GR.getValue('number'));
    }
```

Example-2: addEncodedQuery()

1

Build Query

Incidents New for text Search Grid Split

All > Priority = 1 - Critical .or. Priority = 2 - High > Category = Hardware .or. Category = Software > Created > 2017-01-01 12:00:00 > Short description is not empty

Load Filter Save Filter Add Sort Clear All Run

▼ INCIDENT CONDITIONS

All of these conditions must be met

Priority is 1 - Critical OR AND

Priority is 2 - High OR AND

and

Category is Hardware OR AND

Category is Software OR AND

and

Created after 2017-01-01 12:00:00 OR AND

Short description is not empty OR AND

```
1 var incidentGR = new GlideRecord('incident');
2 incidentGR.addEncodedQuery("priority=1^ORpriority=2^category=hardware^ORcategory=software^sys_created_on>javascript:gs.dateGenerate('2017-01-01','12:00:00')^short_descriptionISNOTEMPTY");
3 incidentGR.query();
```

2

Copy Query

00 > Short description is not empty

Open new window
Copy URL
Copy query

priority=1^ORpriority=2^category=hardware^ORcategory=software^sys_created_on>javascript:gs.dateGenerate('2017-01-01','12:00:00')^short_descriptionISNOTEMPTY

3

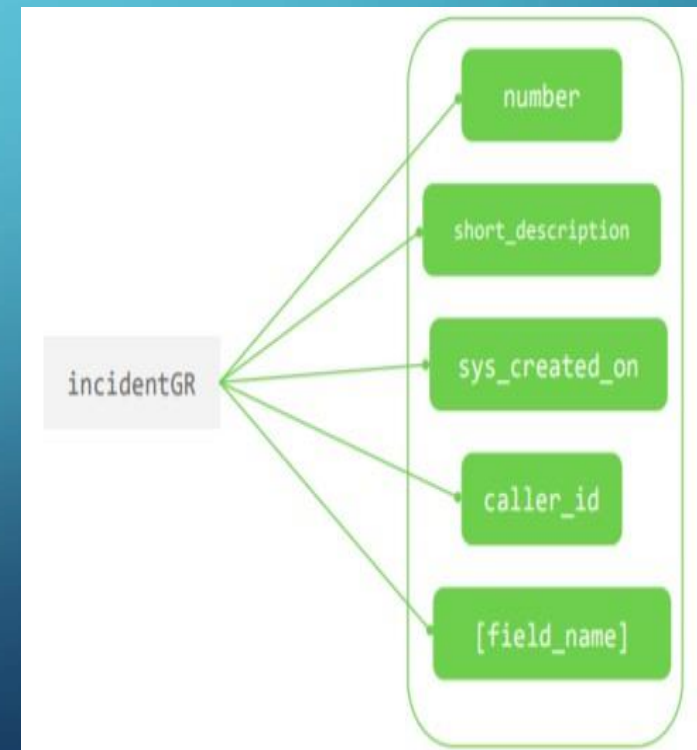
Add to addEncodedQuery()

Accessing a Record's fields :

- Once query() method is executed and stage 2 begins, all fields are just a dot away
- Fields become GlideRecord properties .

Example: If we want the number of incident table having priority 1, it should access the number field.

```
Var incidentGr=new GlideRecord('incident');  
incidentGR.addquery('priority',1);  
incidentGR.query();  
While(incidentGR.next()){  
    gs.print(incidentGR.number);  
}
```



GlideRecord get() Method:

This method accepts either one or two parameters. If only a single parameter is passed in, the method first assumes it is the sys_id of the desired record. If not found, it then tries match the value against the display value. If two parameters are passed in, the **first** is the name of the column within the instantiated GlideRecord to search. The second is the value to search for.

○ **get() method with one parameter:** get(sys_id)

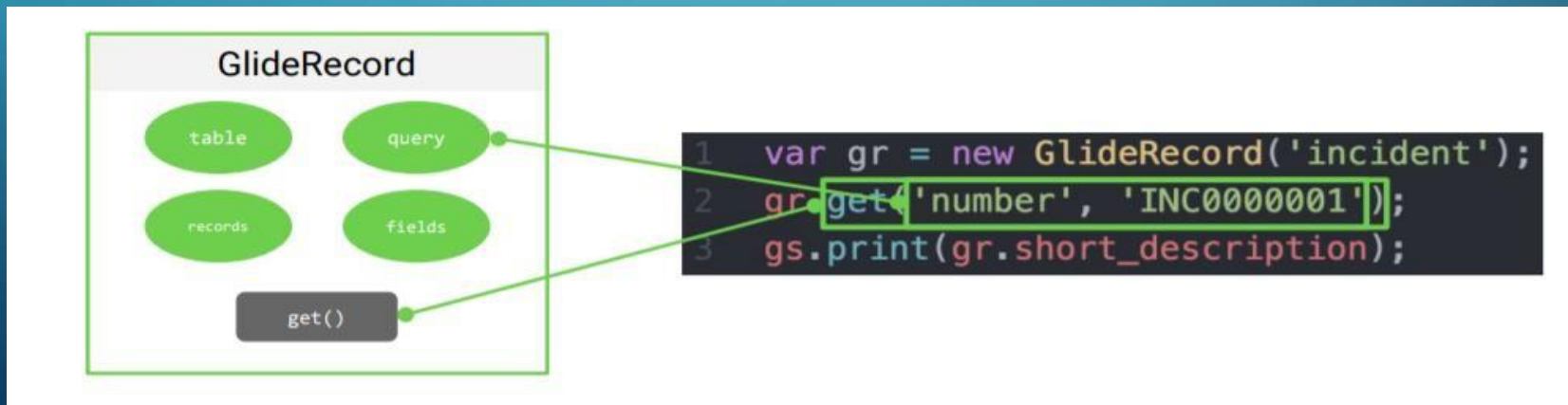
Ex: This example shows how to obtain an incident record by passing in the sys_id.

```
var grIncident = new GlideRecord('incident');  
var returnValue = grIncident.get('a83820b58f723300e7e16c7827bdeed2');  
gs.info(returnValue); // logs true or false  
gs.info(grIncident.short_description); // logs Incident Short description
```

- **get() method with two parameters: get(Object name, Object value)**

Ex: This example shows how to obtain an incident record by passing the field to search (caller_id.name) and the value to match within that field.

```
var grIncident = new GlideRecord('incident');  
var returnValue = grIncident.get('caller_id.name', 'Sahithi sree Yenumulapelly');  
gs.info(returnValue); // logs true or false  
gs.info(grIncident.getValue('number')); // logs Incident Number
```



CRUD GlideRecord:

Create, Read, Update and Delete are GlideRecord queries

`.insert()`

C_{reate}



`.query()*`

R_{ead}



`.update()`

U_{pdate}



`.deleteRecord()`

D_{elete}



C-Create:

To create or insert a record we need to follow the following steps:

- Build GlideRecord
- query()
- newRecord()
- Set field values
- insert()

```
1 // Create
2 var incidentGR = new GlideRecord('incident');
3 incidentGR.query();
4 incidentGR.newRecord();
5 incidentGR.short_description = 'Test 123';
6 incidentGR.insert();
```

Ex: In this example we are creating a new incident with the short description as Test 123.

so first we need to **build a GlideRecord** which means passing the table in which we want to create the records, then actually we need to perform the **query method** that will initialize the GlideRecord object to communicate then we call the **new record method**, this will set all the default values associated with a new record and in fourth step we **set the values** that we would like on this new record. However we can set all of the fields or just a selection of the fields that we would like to have for this new record. Finally we make the insertion so we call the **insert method** that's important because the record isn't actually saved to database yet until we call this method.

Note: Eventhough we call the new record method that object is still in a memory & hasn't been saved to database yet.

R-Read:

To read a record(s) we need to follow an following steps:

➤ Build GlideRecord

➤ Add filter conditions

➤ query()

➤ next()

➤ Print

```
1 // Read
2 var incidentGR = new GlideRecord('incident');
3 incidentGR.addQuery('priority', '1');
4 incidentGR.query();
5 while(incidentGR.next()) {
6     // Read record(s)
7     gs.print(incidentGR.number);
8 }
```

Ex: In this example we are going to read and print the incident numbers which are having 1 as priority.

So first we need to **build a GlideRecord** which means passing the table in which we want to read the records, then actually we need to **add filter condition** to find the priority value which is having 1 using the **addQuery() method**. And we need to perform the **query() method** that will initialize the GlideRecord object to communicate, to read the multiple records we need to perform **next() method** if condition becomes true it helps to iterate until it returns false. Finally print the numbers which are having priority as 1.

U-Update:

Update is used to update or modify the existing values or records. To update an record(s) we need to follow an following steps:

➤ Build GlideRecord

➤ Add filter conditions

➤ query()

➤ next()

➤ Set field values

➤ update()

```
1 // Update
2 var incidentGR = new GlideRecord('incident');
3 incidentGR.addQuery('priority', '1');
4 incidentGR.query();
5 while(incidentGR.next()) {
6     // Update records
7     incidentGR.priority = '2';
8     incidentGR.update();
9 }
```

Ex: In this example we are going to update the priority values as 1 to 2

First we need to **build a GlideRecord** which means passing the table in which we want to update the records, then actually we need to **add filter condition** to find the priority value which is having 1 using the **addQuery() method**.

And we need to perform the **query() method** that will initialize the GlideRecord object to communicate ,to read the multiple records we need to perform **next() method**. And in fifth step we **set the values** that we would like to change the existing record. Finally we call the **update() method** which will update the record in the database.

Similar to insert method the field values are not saved to the database until the update method is called

D-Delete:

Delete operation is used when we want to delete or remove the unwanted record(s) from database. We need to follow following steps to delete an record(s).

➤ Build GlideRecord

➤ Add filter conditions

➤ query()

➤ next()

➤ deleteRecord()

```
1 // Delete
2 var incidentGR = new GlideRecord('incident');
3 incidentGR.addQuery('number', 'INC0000001');
4 incidentGR.query();
5 while(incidentGR.next()) {
6     // Delete record
7     incidentGR.deleteRecord();
8 }
```

Ex: In this example we are deleting the record having the incident number as INC0000001.

First we need to **build a GlideRecord** which means passing the table we want to delete, then actually we need to **add filter condition** to find the **number** value which is having **INC0000001** using the **addQuery() method**. And we need to perform the **query() method** that will initialize the GlideRecord object to communicate, to read the multiple records we need to perform **next() method** it is used to iterate until the condition gets false. Finally we call the **deleteRecord() method** to delete the record from the database(incident table).

GlideAggregate:

GlideAggregate enables you to easily create database aggregation queries.

- Extension of GlideRecord class
- Used when performing aggregate queries (count, min, max, sum, avg)
 - Reports
 - Calculations
- Only works on number fields.

GlideAggregate(String tableName):

Creates a GlideAggregate object on the specified table.

Example:

```
var count = new GlideAggregate('incident');
```

`addAggregate(String agg, String name):`

Adds an aggregate to a database query.

Parameters

Name	Type	Description
agg	String	Name of an aggregate to include in a database query. Valid values: • AVG • COUNT • MIN • MAX • STDDEV • SUM
name	String	Optional. Name of the field to group the results of the aggregation by. Default: Null

Returns

Type	Description
void	Method does not return a value

Example:

```
var incidentGA = new GlideAggregate('incident');
incidentGA.addQuery('category', 'software');
incidentGA.setGroup(false);
incidentGA.addAggregate('COUNT', 'sys_mod_count');
incidentGA.addAggregate('SUM', 'sys_mod_count');
incidentGA.addAggregate('AVG', 'sys_mod_count');
incidentGA.addAggregate('MIN', 'sys_mod_count');
incidentGA.addAggregate('MAX', 'sys_mod_count');
incidentGA.addAggregate('STDDEV', 'sys_mod_count');
incidentGA.query();
if (incidentGA.next()) {
gs.info('COUNT: ' + incidentGA.getAggregate('COUNT', 'sys_mod_count'));
gs.info('SUM: ' + incidentGA.getAggregate('SUM', 'sys_mod_count'));
gs.info('AVG: ' + incidentGA.getAggregate('AVG', 'sys_mod_count'));
gs.info('MIN: ' + incidentGA.getAggregate('MIN', 'sys_mod_count'));
gs.info('MAX: ' + incidentGA.getAggregate('MAX', 'sys_mod_count'));
gs.info('STDDEV: ' + incidentGA.getAggregate('STDDEV', 'sys_mod_count'));
}
```

The background is a blue gradient. In the corners, there are white line-art illustrations of circuit boards or neural network connections, consisting of lines and small circles.

ANY QUERIES?

THANK YOU 😊